

FROM A MYSTERY EXECUTABLE TO A CUSTOM INSTRUMENT

*A Technical Walkthrough — Reverse-Engineering the Ophis App
and Building NATORI-ON-PSYFR, the Predictive Chronology Engine*

Written as a teaching guide — explaining not just what we did,
but why each step made sense, for someone seeing it for the first time.

HOLLYWOOD ROGUE VENTURES

Ophis grammar × Chronicon calendrics · Phoenix 138 · Metonic 19

Contents

Contents 2

What This Document Is..... 3

The Starting Point: a 100 MB Mystery..... 3

Lesson 1 — What an Executable Actually Is 3

Lesson 2 — Peeling the Onion..... 4

Lesson 3 — Understanding How the Program Thinks 4

Lesson 4 — The Plan: Fusion 5

Lesson 5 — The Method That Matters: Verify First 5

Lesson 6 — Building the Engine 6

Lesson 7 — Iterating: Adding the Powerful Features 7

Lesson 8 — A Real Bug, and How We Caught It 7

Lesson 9 — Accessibility and Theming..... 8

Lesson 10 — Branding, the Field Guide, and Safe Renaming..... 8

The Engineering Principles We Practiced 8

Glossary.....10

Appendix — The Toolchain10

What This Document Is

This is a field report and a teaching guide rolled into one. Over several sessions we took a **100-megabyte Windows program a person could not open or understand**, figured out exactly what it was, learned how it worked on the inside, and then used that knowledge to build something new and better: a single-file, offline web application called **NATORI-ON-PSYFR**, plus a companion field guide that teaches people how to use it.

It is written for a **newcomer** — someone who has never reverse-engineered a program, never heard of an “Electron app,” and has only a little experience with code. Every technical term is explained the first time it appears. The goal is not just to list what happened, but to show *why each move was the sensible one*, so you could do the same thing yourself next time.

There are three big ideas to carry through the whole story:

- **Triage before effort.** Before touching a mysterious file, ask the computer what it actually is. The answer usually changes the entire plan.
- **Verify before you build.** We proved the math was correct with automated tests before we wrote a single line of user interface. This is the habit that separates a toy from an instrument.
- **Change the label, not the wiring.** When we rebranded and renamed things, we touched only what users see and left the working machinery alone — so nothing broke.

The Starting Point: a 100 MB Mystery

We began with a single file named `Ophis_v12_Windows.exe`. The owner believed it was something like an Excel or database tool, but it would not open or behave like one, and at roughly **100 megabytes** it was far too large for a simple spreadsheet. A second, much smaller file — `ophis.html` — came alongside it. That pairing turned out to be the first and biggest clue, though we did not know it yet.

The honest starting position was: “We don’t know what this is.” The right response to that is not to guess — it is to investigate methodically.

Lesson 1 — What an Executable Actually Is

An **executable** (the `.exe` on Windows) is a file the computer can run as a program. But “executable” does not tell you *how* it was built or what is inside. The first job is to ask the system to identify it. On Linux, the `file` command inspects the actual bytes and reports what it sees, regardless of the file’s name:

```
$ file Ophis_v12_Windows.exe
Ophis_v12_Windows.exe: PE32 executable (GUI) Intel 80386,
Nullsoft Installer self-extracting archive
```

That one line reframed everything. “**Nullsoft Installer self-extracting archive**” (NSIS) means the file is not really the program — it is an **installer**: a wrapper whose job is to unpack the real program onto your machine. In other words, it is a container, and containers can be opened.

Why 100 MB was a clue, not a problem

A genuine spreadsheet tool is small. When a “simple” app is enormous, the size is usually telling you it ships its own copy of a much larger thing inside it — most often an entire web browser. Hold that thought; it becomes the key to the whole puzzle in the next lesson.

Lesson 2 — Peeling the Onion

Because the installer is just an archive, we can extract it instead of running it. We used [7-Zip](#) (a free archive tool) to list and then unpack the contents:

```
$ 7z l Ophis_v12_Windows.exe      # list what's inside
$ 7z x Ophis_v12_Windows.exe      # extract it
...
$PLUGINS\DIR\app-64.7z           # <- a second archive, nested inside
```

Inside the installer was **another** compressed archive, [app-64.7z](#). We extracted that too, and out came the real application folder. Looking at the files revealed what kind of program this was:

```
Ophis.exe          # the launcher
resources/app.asar  # the actual app, packed in an 'asar' archive
*.dll, locales/, ... # a bundled Chromium (web browser) engine
```

This is an Electron app. Electron is a popular way to build a desktop program by wrapping an ordinary **website** (HTML, CSS, and JavaScript) inside a stripped-down web browser so it runs like a native application. That is what the 100 MB was: the browser engine. And it means the “program” is really a web page in disguise.

The asar archive — and the punchline

The app’s code lives in [app.asar](#), Electron’s simple bundling format. There is an official tool to unpack it. Crucially, Electron apps are **not compiled into unreadable machine code** — the JavaScript inside is the original source, just packaged. “Reverse-engineering” here was really careful unzipping:

```
$ npx @electron/asar extract app.asar ./asar_out
# -> clean, readable source: ophis.html, main.js, src/*.js, lib/*.js
```

Then came the punchline. The [ophis.html](#) bundled inside the giant executable was **byte-for-byte identical** to the little [ophis.html](#) the owner already had — all 39,970 bytes the same. The 100 MB “program” and the tiny web file were the **same application**. The executable was simply that web page wearing a browser as a costume. Everything we needed had been sitting in the small file the whole time.

Lesson 3 — Understanding How the Program Thinks

With readable source in hand, we studied what the app actually does. Ophis is a **date-prediction tool** built on the alternative-chronology research of Jason Breshears (the “Archaix” framework). Stripped to its essence, it works with five ideas:

Term	What it means
X-Dates	Anchor dates you give it — known points in time. Every run needs at least two.
Y	The interval between two anchors, counted in days (“axial rotations” — turns of the Earth).
Operations	Little equations that transform Y into a new offset. Each must begin X1+ or X2+.
Z-Dates	The predictions — the projected dates that come out the other end.
MSRF	A set of “resonant” numbers (such as 138). Outputs that match them are treated as significant.

We also found the program’s helper functions. The most characterful is `oph_flip`, which **reverses a number’s digits** — 138 becomes 831, 19 becomes 91. That single function is what lets the tool chase **palindromes** (sequences that read the same forwards and backwards), a recurring theme in this body of work. Constants like `OPH_PHI` (the golden ratio, 1.618) and `OPH_PI` (3.14) round out the toolkit.

A legal operation, then, looks like this — and the program rejects anything that breaks the grammar:

```
X2+oph_flip(oph_round(Y))      # mirror the interval's digits, project forward
X2+oph_round(Y/138)*138        # snap the interval onto the 138 lattice
Y*2                            # REJECTED - must start with X1+ or X2+
```

Lesson 4 — The Plan: Fusion

Understanding Ophis revealed both its strength and its gap. Its strength is the **prediction grammar** — the machinery that turns intervals into projected dates. Its gap is that it had no rich sense of **what those dates mean**. Separately, the owner had already built an excellent “Chronicon” calendar engine: it knew the Breshears timeline cold — Annus Mundi dating, the 138-year Phoenix cycle, the 19-year Metonic moon cycle, documented historical events, and more.

The plan wrote itself: **keep the Ophis prediction grammar, marry it to the correct Chronicon calendar, and make the fusion the scoring**. Every predicted date would be measured against the real timeline and the meaningful numbers, then ranked. That scoring is the new idea — the thing neither tool did alone.

Lesson 5 — The Method That Matters: Verify First

Here is the single most important practice in the entire project. Before building any buttons or screens, we proved the math was right using small **test harnesses** — short scripts that check

the engine's answers against facts we already know, and report pass or fail. If the foundation is wrong, a beautiful interface just hides the error.

We checked three things, and did not proceed until all three were green:

1. **Chronology anchors (16 of 16 passed).** Does today convert to the expected Annus Mundi 5920 and Long-Count 5138? Does the Great Flood land on AM 1656? Does Petrie's 1882 survey land on AM 5776? Each is a known answer the engine had to reproduce.
2. **Calendar round-trips (812 of 812 passed).** Take a date, convert it to a day-number and back again. If you don't get the original date every single time, the conversions are unsafe. We tested 812 dates; all 812 made the round trip.
3. **Event ledger on the lattice.** Every documented Phoenix event must sit exactly on the 138-year grid (mathematically, "mod 138 equals 108"). All of them did.

A passing harness looks unglamorous, and that is the point — boring output means trustworthy math:

```
$ node core_test.js
OK 2026 -> AM 5920 / LC 5138
OK Great Flood -> AM 1656
OK oph_flip(138) = 831
...
16 / 16 anchor checks passed 812 / 812 round-trips passed
```

Lesson 6 — Building the Engine

Only after the math was trusted did we build the application around it. The design goals were deliberate:

- **Single file.** The whole app — structure, styling, logic, and data — lives in one HTML file. Nothing to install.
- **Air-gapped.** It makes no internet connection. Everything runs locally in the browser, so it works offline and keeps the work private.
- **Faithful but honest.** It presents the Archaix thesis as a worldbuilding and study instrument, not as established science — stated plainly in the app itself.

The interface is organized into clear sections: **The Oracle** (seed anchors, choose operations, and cast predictions), **The Wheels** (read any date across all the chronology cycles), **The Ledger** (the documented events), and **Method** (the honest explanation). The fusion idea shows up as **two scoring lenses** you can switch between live:

Lens	What it rewards	The question it answers
V8 · Chronology-first	Landing on real Phoenix nodes and documented events	Which predictions resonate with history?
V7 · Numbers-first	Palindromes and the signature numbers 138 and 19	Which predictions resonate with the signatures?

The same set of predictions can rank completely differently depending on the lens — which is exactly what makes the tool interesting to explore rather than just read.

Lesson 7 — Iterating: Adding the Powerful Features

With a solid core, we added capability in rounds. Each feature followed the same loop: write it, splice it in with a small build script, then test it before moving on.

Convergence — the real signal

The most valuable addition. **Convergence** finds dates where several *different* operations independently agree. One operation hitting a date is a coincidence; four operations landing together is a pattern. We rank these clusters by “strength” (how many distinct operations agree), and the engine reliably surfaces a strength-4 convergence on the year 2040.

Eclipse overlays — without bloating the file

We extracted real solar and lunar eclipse tables from the original Ophis data (covering roughly 1–3000 CE) so a prediction can be checked against actual sky events. Thousands of eclipses would normally make the file huge, so we used **delta compression** — storing the small gap between each eclipse and the next instead of full dates — shrinking many thousands of records into about 56 kilobytes. A fast lookup then tags any prediction that falls on an eclipse.

Preset packs and save / load

We bundled ready-made operation sets (a 138 pack, a 19-Metonic pack, a Phoenix-lattice pack, a Golden pack) and added the ability to save an entire setup to a file and load it back — the same portable approach as the original tool’s configuration files.

Lesson 8 — A Real Bug, and How We Caught It

Honesty matters in a teaching document, so here is a genuine mistake and the fix. After wiring up the preset buttons, the app failed on load with:

```
Uncaught ReferenceError: Cannot access '$' before initialization
```

The cause is a classic JavaScript trap called the **temporal dead zone**. Our setup code tried to use a helper named `$` before the line that created it had run — like trying to drive a car off the lot before it has been built. The lesson is about **order**: code that runs immediately must not reach for things defined later. The fix was to move the wiring inside the app’s `init()` startup function, so it only runs after everything exists.

More importantly, here is how we caught it: a **runtime test**. Checking that code is grammatically valid (it “parses”) is not the same as checking it actually *runs*. We built a tiny fake browser in the test script so we could start the app in a plain terminal and confirm it initializes with no errors — catching the failure before it ever reached a real user.

```
$ node runtime_test.js
```

```
RUNTIME: init() completed, theme+zoom wiring OK, NO error
```

Lesson 9 — Accessibility and Theming

A good instrument is usable by everyone. We added a **dark / light theme** (it follows your system preference on first visit and remembers your choice), adjustable **text size**, full keyboard navigation, clear focus outlines, descriptive labels for screen readers, and respect for the “reduce motion” setting. None of this changes what the tool does — it changes who can comfortably use it.

Lesson 10 — Branding, the Field Guide, and Safe Renaming

Finally we gave the project its identity and its documentation. The app was renamed to **NATORI-ON-PSYFR**, and we wrote a separate **Field Guide** — a standalone page with a quick-start, a glossary, a cookbook of copy-and-paste formulas (each one validated so none can throw an error), a tag key, worked examples, and a FAQ. The two files link to each other and share theme settings.

The key discipline during renaming: change the label, not the wiring. We updated only what users see — the visible titles, the cross-links, and the names of downloaded files — while deliberately leaving the internal storage keys alone so the two files keep sharing your theme choice. Renaming for looks should never quietly break behavior.

A rename is a one-line, surgical edit when you target it precisely:

```
# update only the download label the user sees, not the logic:
sed -i "s|ophion_config_|Natori-On-PSYFR-config-|" Main-UI.html
```

The Engineering Principles We Practiced

Strip away the specifics and these are the transferable habits — the real takeaways for a newcomer:

- **Triage first.** Ask the computer what a file is before deciding what to do with it. One `file` command saved us from a wrong plan.
- **Verify before building.** Prove the foundation with automated tests, then build on top of proven ground.
- **Test at runtime, not just syntax.** “It parses” is not “it works.” Actually run the thing.
- **Make surgical edits.** Change exactly what you intend, by line, and check that everything else is untouched.
- **Degrade gracefully.** Outside the eclipse data’s range, the app simply omits the tag rather than guessing or crashing.
- **Prefer single-file portability.** One self-contained file is easy to keep, share, and trust.

- **Separate display from logic.** What a user reads and what a program runs are two different layers; change one without disturbing the other.

Glossary

Term	Plain-language meaning
Executable (.exe)	A file the computer can run as a program. Says nothing about how it was built.
NSIS installer	A self-extracting archive whose job is to unpack a program onto your machine.
Archive (7z, zip)	A container holding compressed files. Can be opened and extracted.
Electron app	A desktop program made by wrapping a website inside a bundled web browser.
asar	Electron's simple format for bundling an app's source files together.
Source code	The human-readable instructions a program is written in, before any compiling.
Test harness	A small script that checks a program's output against known answers and reports pass/fail.
Round-trip test	Convert a value out and back; you must get the original, or the conversion is unsafe.
Temporal dead zone	A JavaScript error from using a name before the line that creates it has run.
Air-gapped	Makes no network connection; runs entirely on your own machine.
Delta compression	Storing the gap between values instead of full values, to save space.
Convergence	When several different operations independently agree on the same date.
Palindrome	A sequence that reads the same forwards and backwards (e.g., 138–831).

Appendix — The Toolchain

Every tool used in this project, and the one job it did for us:

Tool	What we used it for
<code>file</code>	Identify what the mystery .exe actually was.
<code>7z</code>	Extract the NSIS installer and the nested app archive.
<code>@electron/asar</code>	Unpack app.asar into readable JavaScript source.
<code>node</code>	Run the test harnesses and the runtime checks.
<code>sed</code>	Make precise, single-line text edits for renaming and re-branding.
<code>docx</code>	Generate this very document.

That is the whole arc: from a file nobody could open to a verified, documented, accessible instrument — **built on proof, renamed with care, and explained so the next person can follow the same path.**